



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка кроссплатформенных приложений дополненной
реальности»

Практическая работа № 4

Студент группы *ИКБО-31-24, Линьков М.В.*

(подпись)

Преподаватель *Колодочкин А.А.*

(подпись)

Отчет представлен «__» _____ 2026 г.

Москва 2026

Введение

Цель работы: разработка WebAR-приложения на Three.js с использованием WebXR Device API для пространственного позиционирования и фиксации 3D-объектов на плоскости пола через механизм hit-test.

В рамках выполнения практической работы решаются следующие задачи:

- инициализировать 3D-сцену Three.js и настроить запуск сессии WebXR immersive-ar;
- сконфигурировать пространство local-floor и источник зондирующего луча XRHitTestSource;
- разработать кольцевой маркер (reticle) для визуализации точки пересечения с поверхностью;
- реализовать фиксацию 3D-объекта в мировых координатах сцены по тапу на экран;
- протестировать устойчивость пространственного удержания объекта при перемещении камеры в HTTPS.

1 Реализация WebAR-сценария

Для выполнения практической работы разработано WebAR-приложение на базе Three.js и WebXR Device API. При запуске сессии immersive-ar с параметром hit-test инициализируется система отсчета local-floor и запрашивается источник зондирующего луча XRHitTestSource в пространстве наблюдателя (viewer). Каждый кадр в цикле анимации выполняется опрос пересечений с физической поверхностью (frame.getHitTestResults). Матрица трансформации кольцевой ретикулы обновляется на основе полученной пространственной позы (hit.getPose), а при получении события select контроллера трехмерный объект инстанцируется в точке касания и надежно закрепляется в глобальной системе координат сцены (листинг 1).

Листинг 1 – Реализация WebAR-сценария практической работы №4

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
  <meta charset="UTF-8">
  <title>Three.js WebXR Hit-test (П4)</title>
  <style>
    body { margin: 0; overflow: hidden; background: #111; }
    #ar-btn { position: fixed; bottom: 20px; left: 50%; transform: translateX(-50%); z-index: 20;
padding: 12px 24px; font-weight: bold; background: #007bff; color: #fff; border: none; border-radius:
8px;
  }
</style>
  <script src="https://cdn.jsdelivr.net/npm/three.js@r128/three.min.js"></script>
</head>
<body>
  <button id="ar-btn">Войти в AR</button>
  <script>
    const params = new URLSearchParams(window.location.search), demo = params.get("demo"), arBtn =
document.getElementById("ar-btn");
    let camera, scene, renderer, controller, reticle, hitTestSource = null, localSpace = null;

    function init() {
      scene = new THREE.Scene(); camera = new THREE.PerspectiveCamera(70, window.innerWidth /
window.innerHeight, 0.01, 20);
      const light = new THREE.HemisphereLight(0xffffff, 0xbbbfff, 1); light.position.set(0.5, 1,
0.25);
      renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
      renderer.setPixelRatio(window.devicePixelRatio);
      renderer.setSize(window.innerWidth, window.innerHeight); renderer.xr.enabled = true;
      document.body.appendChild(renderer.domElement);
      reticle = new THREE.Mesh(new THREE.RingGeometry(0.12, 0.15, 32).rotateX(-Math.PI / 2), new
```

```

THREE.MeshBasicMaterial({ color: 0x00ff88, side: THREE.DoubleSide }));
reticle.matrixAutoUpdate = false; reticle.visible = false; scene.add(reticle);
controller = renderer.xr.getController(0);
controller.addEventListener("select", () => {
  if (reticle.visible) { const p = new THREE.Vector3(), q = new THREE.Quaternion(), s =
new THREE.Vector3(); reticle.matrix.decompose(p, q, s); spawnObject(p, q); }
});
scene.add(controller);
window.addEventListener("resize", () => { camera.aspect = window.innerWidth /
window.innerHeight; camera.updateProjectionMatrix(); renderer.setSize(window.innerWidth,
window.innerHeight); });
if (demo) setupDemo(demo); else setupWebXR();
renderer.setAnimationLoop(render);
}
function setupDemo(mode) {
  arBtn.style.display = "none"; camera.position.set(0, 1.2, 1.5); camera.lookAt(0, 0, -1.2);
scene.add(new THREE.GridHelper(10, 20, 0x00ff88, 0x333333));
reticle.visible = true; reticle.position.set(0, 0.01, -1.2); reticle.updateMatrix();
if (mode === "placed") spawnObject(new THREE.Vector3(0, 0, -1.2));
}
function setupWebXR() {
  if (!("xr" in navigator)) return arBtn.disabled = true;
  navigator.xr.isSessionSupported("immersive-ar").then(sup => arBtn.disabled = !sup);
  arBtn.addEventListener("click", async () => {
    const session = await navigator.xr.requestSession("immersive-ar", { requiredFeatures:
["hit-test"] });
    renderer.xr.setReferenceSpaceType("local-floor"); await
    arBtn.style.display = "none";
    const viewer = await session.requestReferenceSpace("viewer"); hitTestSource = await
    session.requestHitTestSource({ space: viewer });
    localSpace = await session.requestReferenceSpace("local-floor");
    session.addEventListener("end", () => { hitTestSource = null; arBtn.style.display =
"block"; });
  });
}
function spawnObject(p, q) {
  const mesh = new THREE.Mesh(new THREE.CylinderGeometry(0.1, 0.1, 0.25, 32), new
THREE.MeshStandardMaterial({ color: 0x3388ff, roughness: 0.3, metalness: 0.2 }));
  mesh.position.copy(p); if (q) mesh.quaternion.copy(q); mesh.position.y += 0.125;
scene.add(mesh);
}
function render(t, frame) {
  if (frame && hitTestSource && localSpace) {
    const hits = frame.getHitTestResults(hitTestSource); reticle.visible = hits.length >
0;
    if (hits.length > 0)
      reticle.matrix.fromArray(hits[0].getPose(localSpace).transform.matrix);
    renderer.render(scene, camera);
  }
}
</script>
</body>
</html>

```

Разделение пространств отсчета local-floor и viewer обеспечивает математически строгое сопоставление координат луча камеры с реальной геометрией окружения. Визуальный маркер reticle ориентируется параллельно плоскости пола поворотом геометрии RingGeometry на -90° по оси X. При фиксации объекта трехмерный меш переносится в глобальное дерево сцены scene с учетом смещения высоты на половину геометрического габарита, что исключает эффект проваливания модели под плоскость пола.

2 Результат работы

Тестирование разработанного WebAR-приложения проводилось в защищенном HTTPS/localhost-контексте веб-браузера на мобильном устройстве и в среде программной симуляции SwiftShader. Проверены переход состояний при поиске плоскости пола, точность совмещения маркера ретикулы с точкой падения луча и стабильность фиксации цилиндрического 3D-объекта при изменении ракурса наблюдения. Результаты тестирования представлены на рисунках 1 и 2.

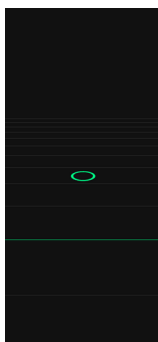


Рисунок 1 – Позиционирование кольцевого маркера (ретикулы) на обнаруженной поверхности пола

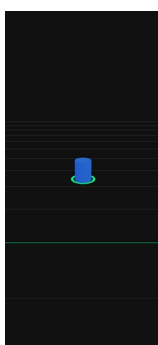


Рисунок 2 – Фиксация 3D-объекта на плоскости и сохранение пространственного положения

Проведенные инструментальные испытания подтвердили, что алгоритм WebXR Hit-test стабильно определяет горизонтальную плоскость пола и транслирует матрицу попадания в маркер. После размещения по событию тапа 3D-объект надежно удерживается в системе координат local-floor без видимого дрейфа, вибраций и задержек при перемещении камеры.

Вывод

В ходе практической работы №4 разработана WebAR-сцена на базе Three.js с поддержкой сессий WebXR immersive-ar. Настроен механизм зондирования поверхностей XRHitTestSource в системе координат local-floor, визуализирована динамическая кольцевая ретикула и реализовано размещение 3D-объектов по событию тапа (select). Испытания подтвердили корректность математической трансформации поз и устойчивую фиксацию виртуального объекта в пространстве.